

LESSON 4

Strings

Immutable sequences. Every modification creates a new string.

4.1 String Reversal (Multiple Ways)

4.2 Valid Palindrome

4.3 Longest Substring Without Repeating Chars

4.4 String Compression

4.5 String Rotation Check

4.6 Practice Problems

4.1 String Reversal

```
# Method 1: Slicing (fastest, most Pythonic)
"hello"[::-1]    # "olleh"

# Method 2: Two Pointers (show understanding in interview)
def reverse_string(s):
    chars = list(s)          # string -> list (mutable)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return "".join(chars)    # list -> string

# Method 3: Built-in
"".join(reversed("hello"))  # "olleh"
```

4.2 Valid Palindrome

```
# Ignore spaces, punctuation, case
# "A man, a plan, a canal: Panama" -> True

def is_valid_palindrome(s):
    cleaned = "".join(c.lower() for c in s if c.isalnum())
    left, right = 0, len(cleaned) - 1
    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True

# One-liner shortcut:
def is_valid_palindrome(s):
    c = "".join(x.lower() for x in s if x.isalnum())
    return c == c[::-1]
```

```
"A man, a plan, a canal: Panama"
After cleaning: "amanaplanacanalpanama"
L='a', R='a' -> match
L='m', R='m' -> match
... all match ... -> True
```

4.3 Longest Substring Without Repeating Characters

Combines **Sliding Window + Hash Map**. Very commonly asked.

```
# Input: "abcabcbb"   Output: 3 ("abc")
# Input: "bbbbbb"     Output: 1 ("b")

def length_of_longest_substring(s):
    seen = {}          # char -> last index seen
    left = 0           # window start
    max_len = 0

    for right in range(len(s)):
        # If char already in window, shrink from left
        if s[right] in seen and seen[s[right]] >= left:
            left = seen[s[right]] + 1

        seen[s[right]] = right    # update last seen position
        max_len = max(max_len, right - left + 1)

    return max_len
```

```
s = "abcabcbb"

r=0 'a': seen={a:0}, window [a], len=1
r=1 'b': seen={a:0,b:1}, window [ab], len=2
r=2 'c': seen={a:0,b:1,c:2}, win [abc], len=3
r=3 'a': a seen at 0 >= left(0), left=1
        seen={a:3,b:1,c:2}, win [bca], len=3
r=4 'b': b seen at 1 >= left(1), left=2
        seen={a:3,b:4,c:2}, win [cab], len=3
r=5 'c': c seen at 2 >= left(2), left=3
        seen={a:3,b:4,c:5}, win [abc], len=3
...
Answer: 3
```

4.4 String Compression

```
# "aabcccccaaa" -> "a2b1c5a3"
# Return original if compressed isn't shorter

def compress(s):
    if not s: return s
    result = []
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(s[i-1] + str(count))
            count = 1

    result.append(s[-1] + str(count)) # don't forget last group!
    compressed = "".join(result)
    return compressed if len(compressed) < len(s) else s
```

```

"aabcccccaaa"
i=1: a==a, count=2
i=2: b!=a, append "a2", count=1
i=3: c!=b, append "b1", count=1
i=4: c==c, count=2
i=5: c==c, count=3
i=6: c==c, count=4
i=7: c==c, count=5
i=8: a!=c, append "c5", count=1
i=9: a==a, count=2
i=10: a==a, count=3
End: append "a3"
Result: "a2b1c5a3" (8 < 11) -> return compressed

```

4.5 More String Problems

```

# Check if all characters unique
def all_unique(s):
    return len(set(s)) == len(s)

# Check if s2 is rotation of s1
# "waterbottle" is rotation of "erbottlewat"
def is_rotation(s1, s2):
    return len(s1) == len(s2) and s2 in (s1 + s1)
# Why? "waterbottlewaterbottle" contains "erbottlewat"

# Reverse words in sentence
# "the sky is blue" -> "blue is sky the"
def reverse_words(s):
    return " ".join(s.split()[::-1])

# Count vowels and consonants
def count_vc(s):
    vowels = set("aeiouAEIOU")
    v = sum(1 for c in s if c in vowels)
    c = sum(1 for c in s if c.isalpha() and c not in vowels)
    return v, c

```